Titlu: Proiect 2 - Break the Login, Atacarea și securizarea autentificării

Student: Alexandru Pătrașcu

Curs: Dezvoltarea Aplicațiilor Software Securizate Universitatea

INTRODUCERE

- Scopul acestui proiect este înțelegerea practică a modului în care un sistem de autentificare este atacat în mediul real și, ulterior, implementarea corectă a controalelor de securitate necesare pentru a rezista acestor amenințări.

- Scenariul presupune auditarea și securizarea aplicației interne a unei companii fictive denumite AuthX. Această aplicație este utilizată de angajați pentru accesul la resurse sensibile, motiv pentru care mecanismul de autentificare reprezintă un punct critic de securitate.

- În cadrul acestui proiect am asumat un rol dublu, acționând inițial ca un specialist în testarea securității pentru a identifica și demonstra vulnerabilitățile, iar ulterior ca un dezvoltator software pentru a remedia problemele găsite și a întări sistemul.

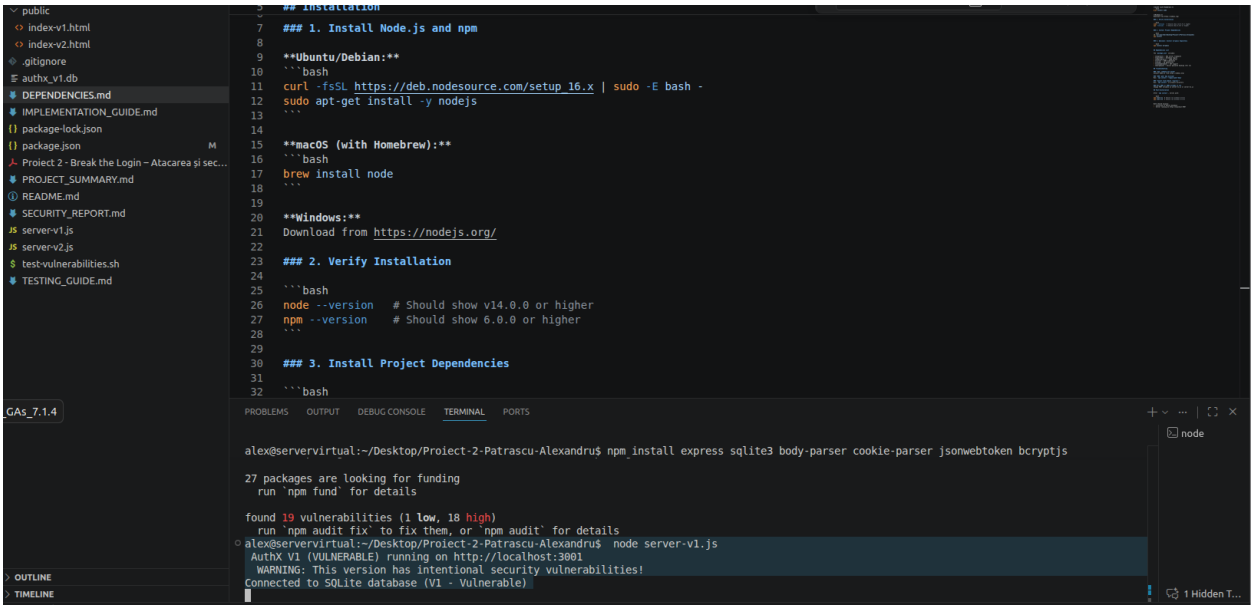
- Din punct de vedere arhitectural, aplicația a fost dezvoltată folosind limbajul JavaScript pe platforma Node.js, utilizând framework-ul Express pentru gestionarea cererilor HTTP. Baza de date aleasă este SQLite, aceasta fiind o soluție optimă pentru demonstrații locale care respectă structura relațională necesară, incluzând tabele pentru utilizatori, tichete de suport și jurnale de audit.

Setup mediu

- Pentru a asigura un mediu de testare izolat și reproductibil, similar cu un mediu de producție real, toate operațiunile au fost desfășurate într-o mașină virtuală. Am configurat un sistem de operare bazat pe kernel Linux, utilizând mediul de virtualizare Oracle VirtualBox.

- În cadrul acestui mediu virtual, am instalat platforma Node.js și managerul de pachete aferent. De asemenea, am inițializat proiectul și am instalat modulele necesare funcționării, printre care se numără express, sqlite3 pentru comunicarea cu baza de date, body-parser pentru extragerea datelor din cereri și bcryptjs pentru securizarea viitoare a parolelor.

- După finalizarea instalării dependențelor, am rulat prima versiune a aplicației, cea vulnerabilă, folosind comanda de pornire a serverului. Conexiunea la baza de date SQLite a fost realizată cu succes, iar tabelele au fost generate automat la prima rulare.



Implementare

- Produsul minim viabil cuprinde funcționalitățile de bază necesare unui sistem de management al accesului. Logica de autentificare se bazează pe rute specifice pentru înregistrare, conectare, deconectare și resetare a parolei. Înregistrarea utilizatorului preia un nume de utilizator, o adresă de email și o parolă, pe care le stochează în tabelul utilizatorilor alături de un rol predefinit.

- Conectarea verifică aceste credențiale și, în caz de succes, emite un identificator de sesiune. Acest identificator este stocat în baza de date și trimis clientului sub forma unui cookie, menținând astfel starea de autentificare între cererile HTTP ulterioare. Deconectarea șterge acest identificator din baza de date și invalidează cookie-ul.

- Pentru a îndeplini cerințele legate de o bază de date cu un grad mediu de complexitate, am implementat și structura tabelului pentru tichete, care include un identificator pentru proprietar, stabilind astfel o relație cheie externă cu tabelul

utilizatorilor. Această structură previne vulnerabilitățile de tip referință directă nesigură la obiect, deoarece permite interogarea tichetelor doar pe baza identificatorului utilizatorului curent din sesiune.

- De asemenea, am creat un tabel dedicat jurnalelor de audit, care înregistrează acțiunile critice precum tentativele de autentificare eșuate, conectările reușite și cererile de resetare a parolei, incluzând adresa IP a clientului pentru a asigura trasabilitatea.

Prezentare vulnerabilități

- Analiza primei versiuni a codului a scos la iveală șase vulnerabilități critice, care pot fi mapate pe standardele din industrie .

- În primul rând, politica de parole este extrem de permisivă. Sistemul acceptă parole de doar trei caractere și nu impune nicio restricție de complexitate, ceea ce face conturile susceptibile la atacuri bazate pe dicționare. Această deficiență se încadrează la eșecuri de identificare și autentificare.

- În al doilea rând, stocarea parolelor în baza de date se face utilizând funcția de dispersie MD5. Această funcție este considerată nesigură și a fost depreciată din motive criptografice, deoarece permite atacatorilor să recupereze parolele foarte rapid prin utilizarea tabelelor precalculate.

- A treia vulnerabilitate constă în lipsa unui mecanism de limitare a ratei de cereri. Un atacator poate trimite un număr infinit de cereri de autentificare într-un timp foarte scurt, facilitând atacurile de tip forță brută.

- A patra problemă identificată este enumerarea utilizatorilor. Sistemul întoarce un anumit mesaj de eroare dacă utilizatorul nu există și un alt mesaj dacă parola este incorectă. Această diferențiere permite unui atacator să descopere ce nume de utilizator sunt valide în sistem.

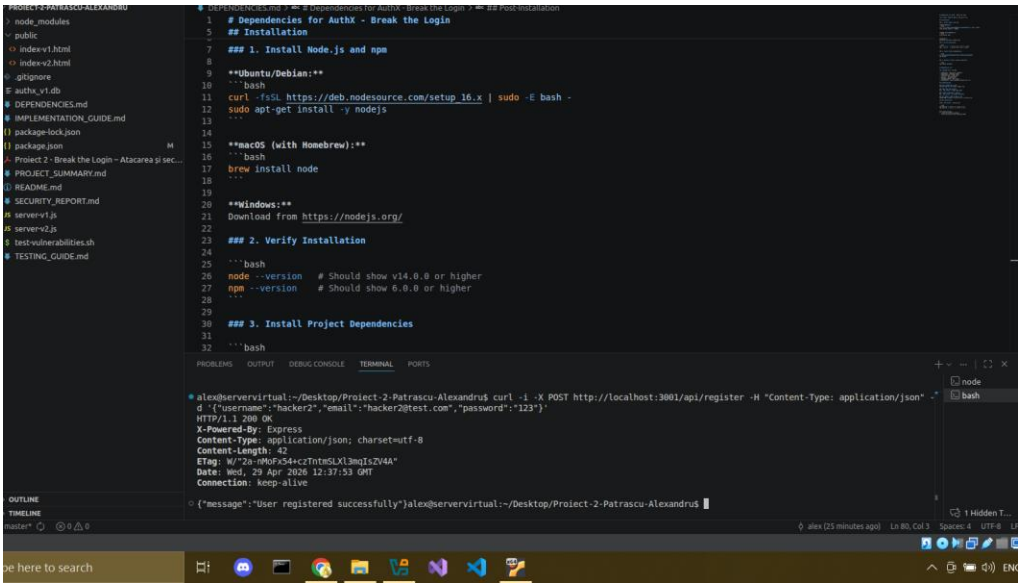
- A cincea vulnerabilitate este legată de gestionarea nesigură a sesiunilor. Cookie-ul emis de aplicație nu posedă atribute de securitate care să interzică accesul din scripturi sau care să forțeze transmisia doar prin conexiuni criptate. Mai mult, perioada de valabilitate a sesiunii este foarte lungă, crescând fereastra de oportunitate pentru furtul de sesiune.

- Nu în ultimul rând, mecanismul de resetare a parolei folosește un identificator predictibil, reprezentat pur și simplu de numărul de ordine al utilizatorului din baza de date. Acest identificator nu expiră niciodată și nu este invalidat după prima utilizare, permițând compromiterea oricărui cont.

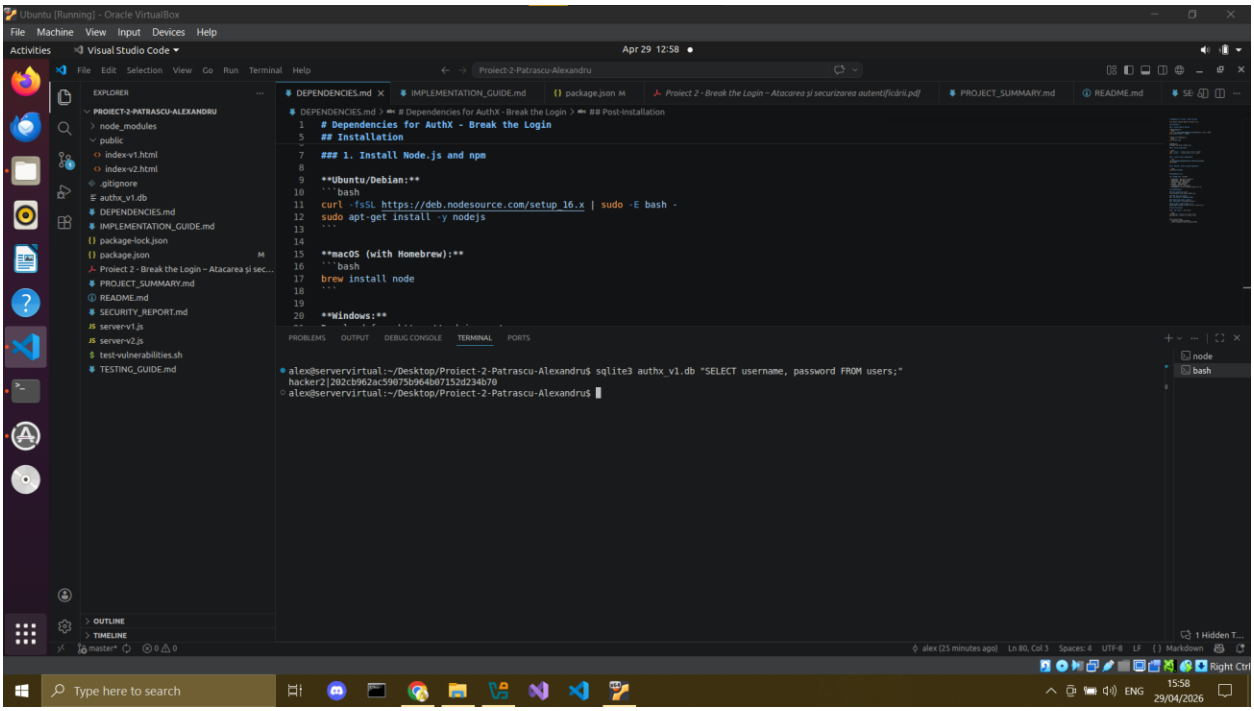
Demonstrare atac

- Pentru a valida existența acestor vulnerabilități, am efectuat o serie de teste practice direct din terminal, simulând comportamentul unui atacator prin interceptarea și manipularea cererilor HTTP.

- Am început prin a demonstra politica slabă de parole. Am trimis o cerere de înregistrare către server, specificând un nume de utilizator fictiv și o parolă formată din doar trei cifre. Sistemul a acceptat cererea fără a ridica nicio eroare de validare a complexității, confirmând crearea contului.

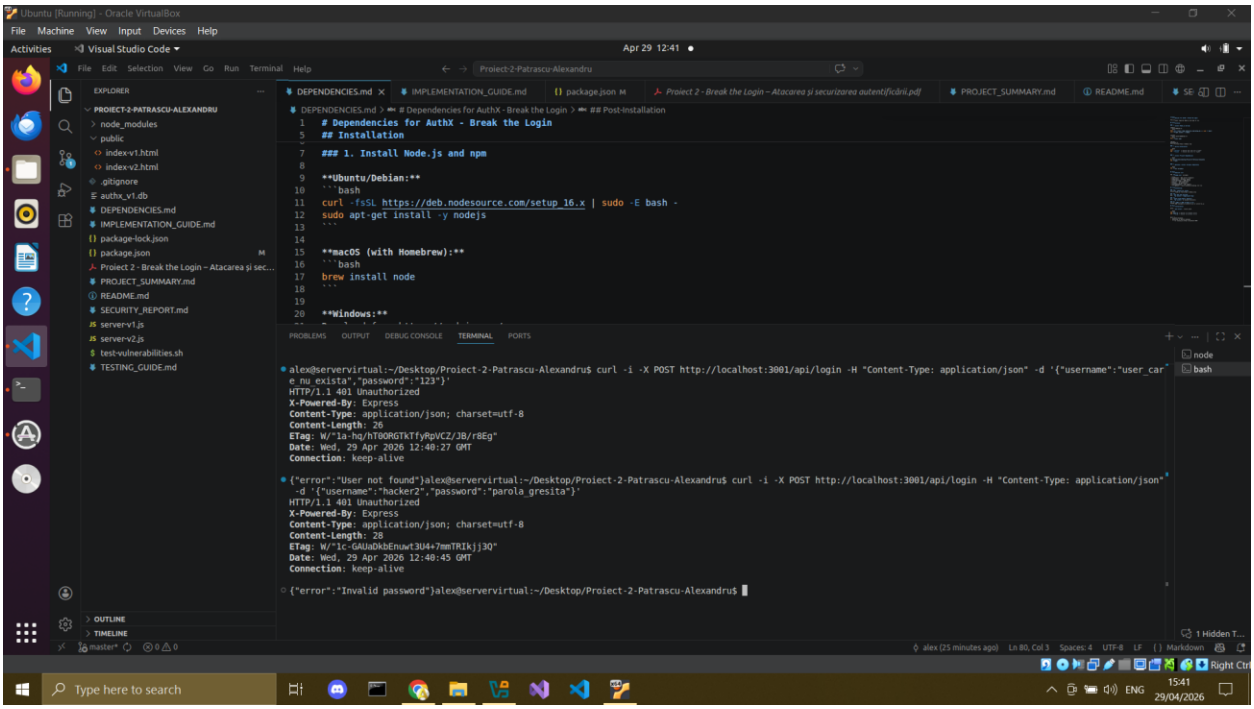


- Mai departe, am simulat scenariul în care un atacator ar obține acces la fișierul bazei de date. Prin interogarea directă a tabelului de utilizatori din terminal, am putut vizualiza formatul în care sunt stocate parolele. Am observat că acestea sunt reprezentate sub forma unui șir specific algoritmului MD5, care poate fi decriptat aproape instantaneu folosind resurse gratuite disponibile pe internet.

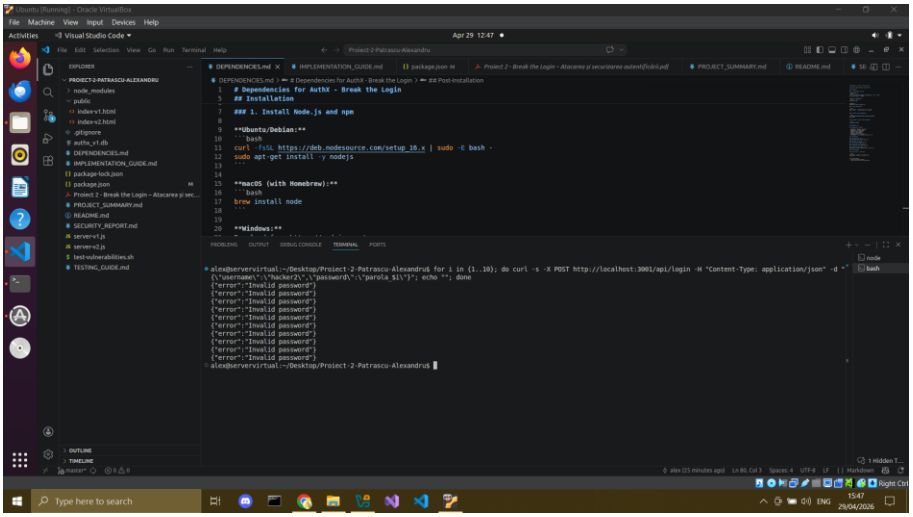


- Următorul pas a fost demonstrarea vulnerabilității de enumerare a utilizatorilor. Am trimis o cerere de autentificare folosind un nume de utilizator despre care știam sigur că nu există în baza de date.

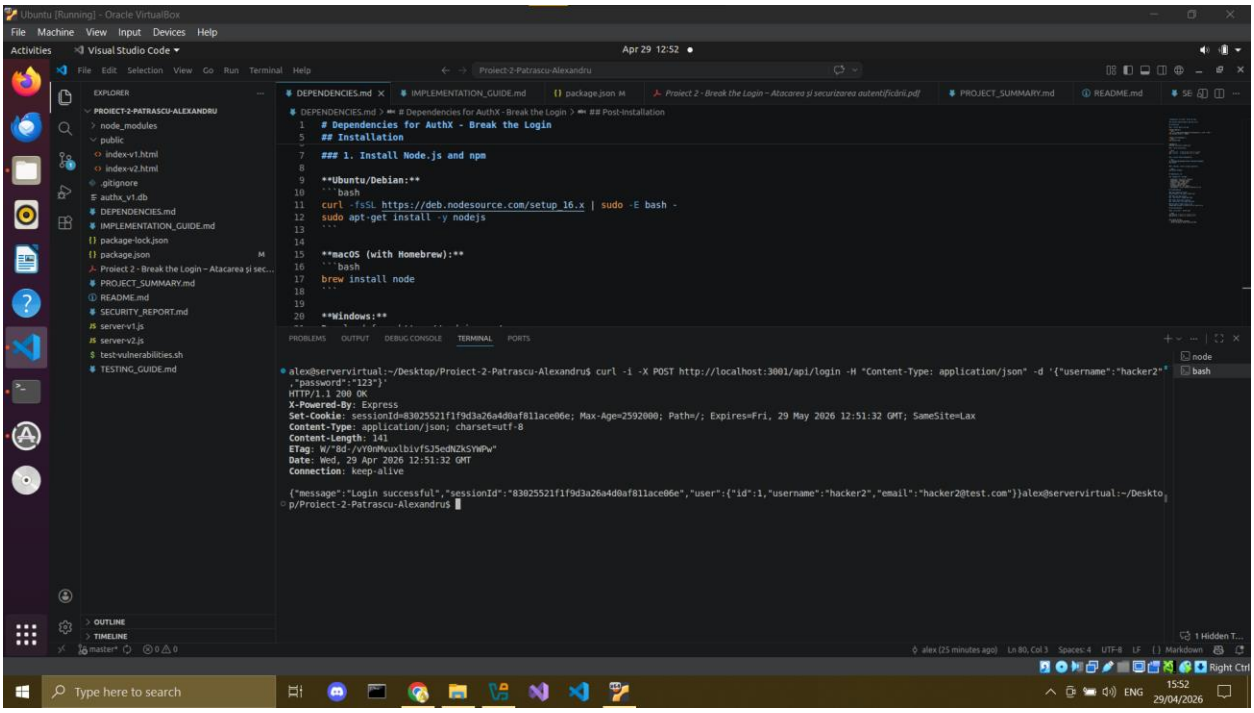
- Răspunsul serverului a fost o eroare care preciza clar lipsa acelui utilizator. Imediat după, am trimis o nouă cerere folosind contul creat anterior, dar intenționat cu o parolă greșită. De această dată, mesajul primit a indicat că parola este invalidă, confirmând practic că numele de utilizator trimis este corect și înregistrat în sistem.



- Pentru a demonstra lipsa limitării ratei de cereri, am construit un scurt script în terminal care a trimis cereri succesive de autentificare eșuată într-o buclă repetitivă. Serverul a procesat fiecare cerere în parte, întorcând eroarea de parolă invalidă de fiecare dată, dar fără a bloca temporar contul și fără a întrerupe conexiunea, permițând continuarea atacului la infinit.

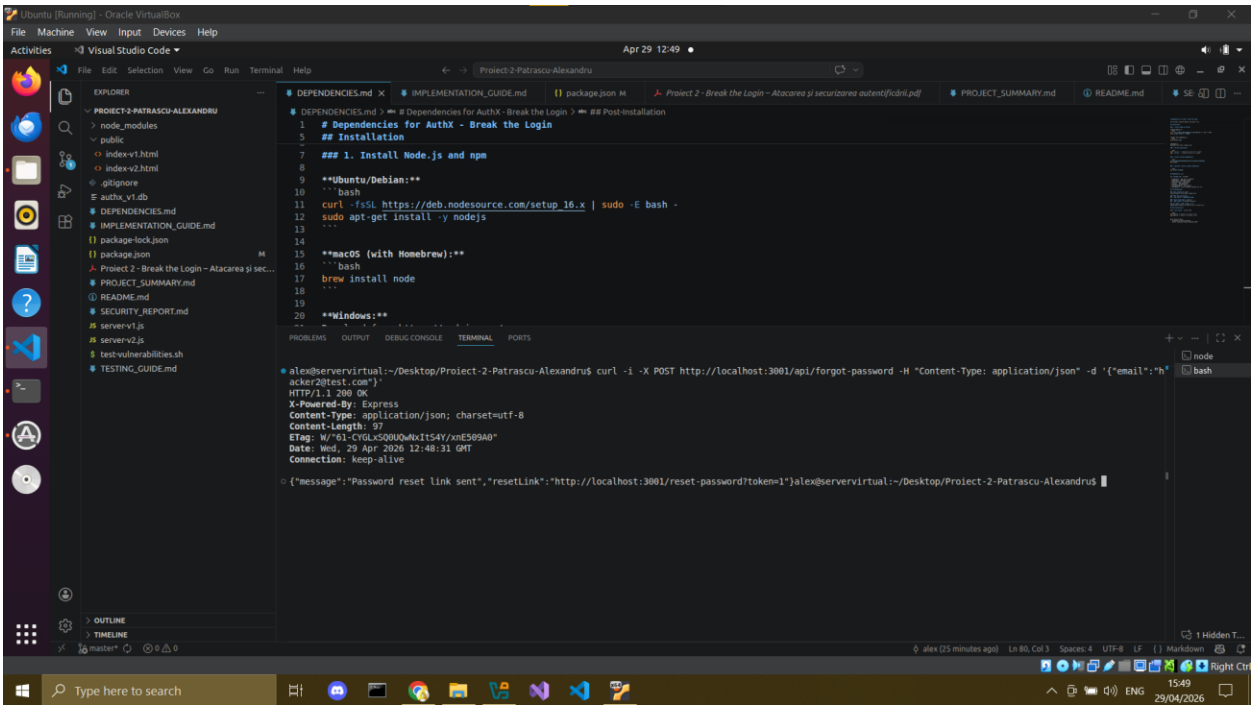


- În ceea ce privește gestionarea sesiunilor, am realizat o autentificare validă și am analizat anteturile răspunsului HTTP primit de la server. Am identificat directivarea de setare a cookie-ului și am constatat vizual lipsa flag-urilor de securitate esențiale, precum și valoarea neobișnuit de mare setată pentru perioada de expirare a sesiunii.



- Ultimul atac demonstrat vizează mecanismul de resetare a parolei. Am inițiat o cerere de uitare a parolei pentru adresa de email a contului țintă.

- Serverul a răspuns expunând direct linkul de resetare, în cadrul căruia identificatorul unic de securitate era pur și simplu cifra unu. Această predictibilitate absolută demonstrează că un atacator nu are nevoie să intercepteze e-mailul victimei, ci poate deduce identificatorul și prelua controlul oricărui cont.



Analiză impact

- Impactul cumulat al acestor vulnerabilități este devastator pentru organizație. Utilizarea unor parole slabe, combinată cu o stocare nesigură, înseamnă că o potențială compromitere a bazei de date va duce la aflarea rapidă a tuturor parolelor în clar.

- Acest lucru permite atacatorilor să folosească aceste credențiale în alte sisteme, declanșând compromiteri în lanț. Absența unei limitări a ratei de acces și prezența erorilor de enumerare a utilizatorilor transformă atacurile de forță brută dintr-un risc teoretic într-o certitudine operațională.

- Furtul de sesiune facilitează accesul direct la datele utilizatorului fără necesitatea decriptării parolei, iar tokenul predictibil pentru resetarea parolei oferă o cale banală de a bloca accesul proprietarilor legitimi la propriile lor conturi.

Implementare fix

- Pentru a remedia aceste probleme grave, am dezvoltat o a doua versiune a aplicației, aplicând principii de securitate riguroase.

- Am rescris logica de înregistrare pentru a include o funcție robustă de validare. Această funcție impune o lungime minimă a parolei de douăsprezece caractere și obligă utilizatorul să includă litere majuscule, litere minuscule, cifre și caractere speciale.

- Pentru protejarea la nivel de stocare, am înlocuit vechiul algoritm de criptare cu algoritmul bcrypt. Acesta generează automat o valoare unică de perturbare pentru fiecare parolă și utilizează un factor de cost care încetinește intenționat procesul de calculare, făcând atacurile bazate pe tabele precalculate ineficiente.

- Pentru a opri atacurile de tip forță brută, am implementat un sistem dublu de protecție.

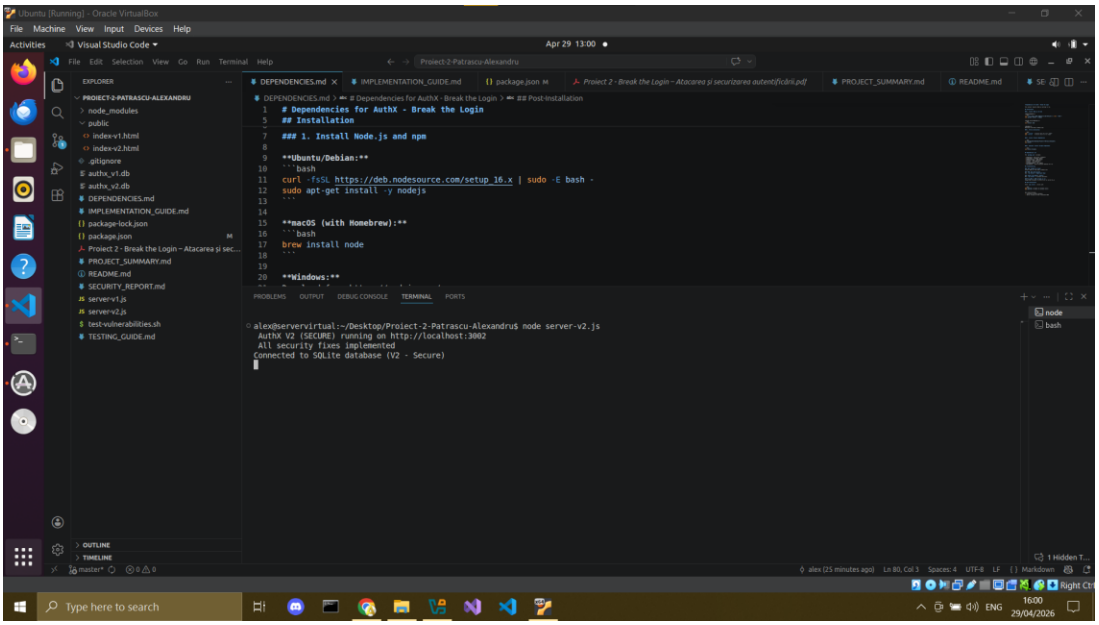
- Primul nivel folosește o structură de date în memoria aplicației pentru a contoriza cererile de conectare efectuate într-un interval de cincisprezece minute.

- Al doilea nivel acționează la nivelul bazei de date, numărând autentificările eșuate specifice fiecărui utilizator. La atingerea pragului de cinci încercări greșite, contul este blocat automat, refuzând orice altă cerere până la expirarea perioadei de penalizare.

- În plus, pentru a combate enumerarea, am modificat blocurile condiționale din cod astfel încât să returneze același text generic de eroare, indiferent dacă eșecul a fost cauzat de un utilizator inexistent sau de o parolă greșită. De asemenea, am introdus o mică întârziere aleatorie în răspunsul serverului pentru a preveni deducerile bazate pe timpul de execuție al interogărilor.

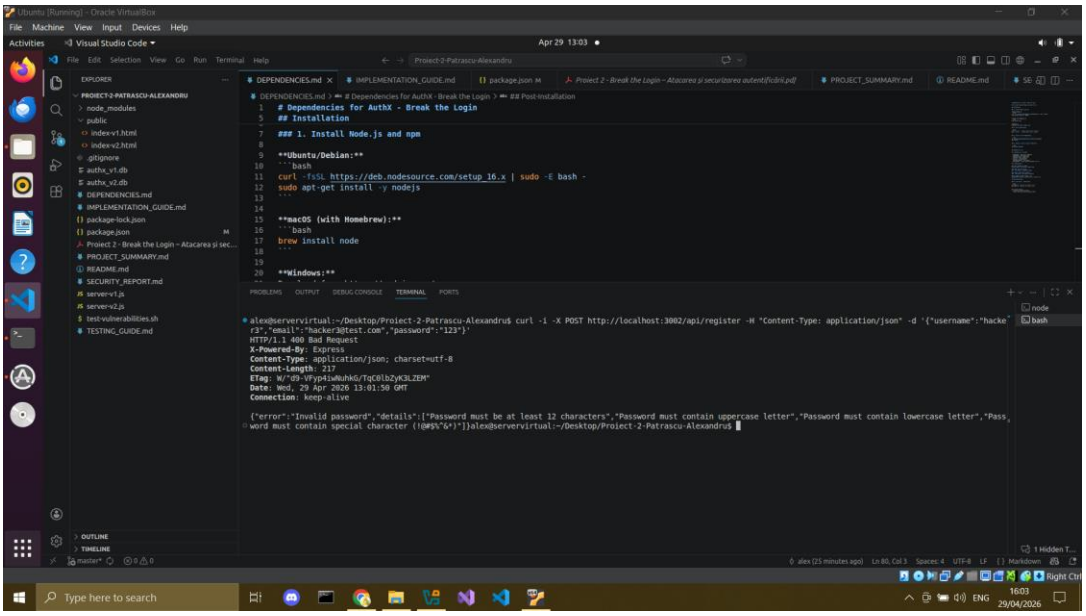
- În privința sesiunilor, am reconfigurat anteturile răspunsurilor HTTP. Identificatorii de sesiune sunt acum generați printr-o funcție criptografică sigură, iar cookie-urile sunt marcate strict pentru a nu putea fi accesate de scripturile din pagină și pentru a fi transmise exclusiv peste conexiuni sigure. Timpul de expirare a fost redus drastic la o singură oră.

- Pentru resetarea parolei, noul mecanism generează o secvență binară lungă, complet aleatorie. Această secvență este salvată în baza de date cu o dată de expirare fixă de cincisprezece minute și include o coloană dedicată care marchează utilizarea. Odată ce o parolă este schimbată cu succes, acel identificator de resetare devine imediat invalid pentru cererile viitoare.

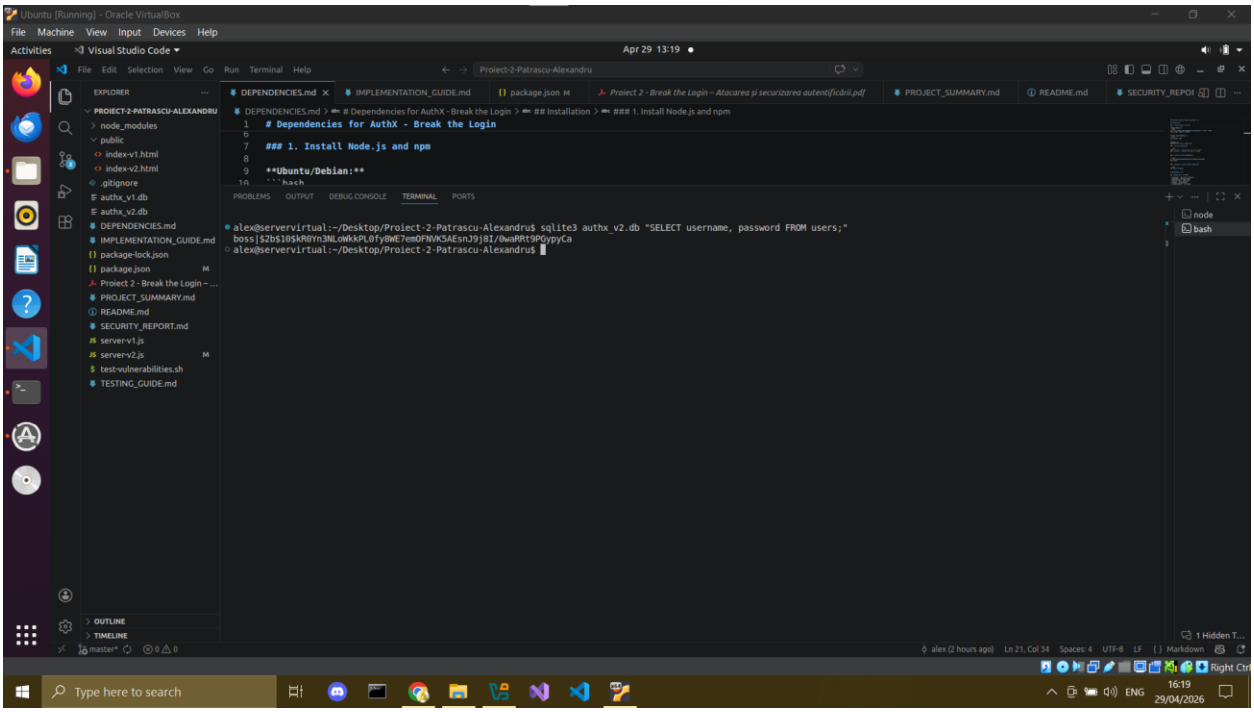


Re-test

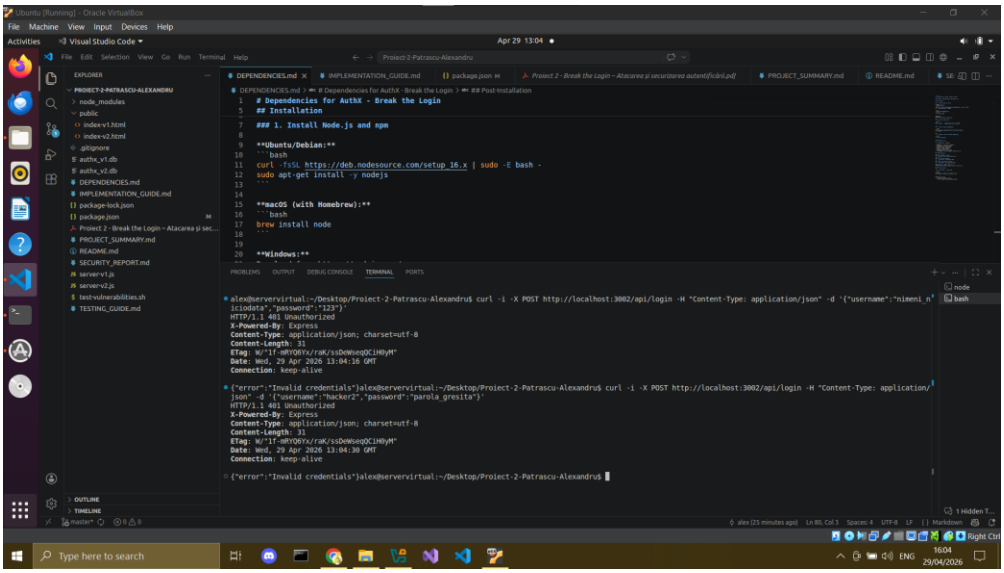
- Odată implementate aceste controale, am reluat seria de teste ofensive pentru a demonstra validitatea protecțiilor. Am încercat să creez un nou cont utilizând aceeași parolă scurtă din primul test. Serverul a respins cererea, detaliind exact care reguli de complexitate nu au fost îndeplinite.



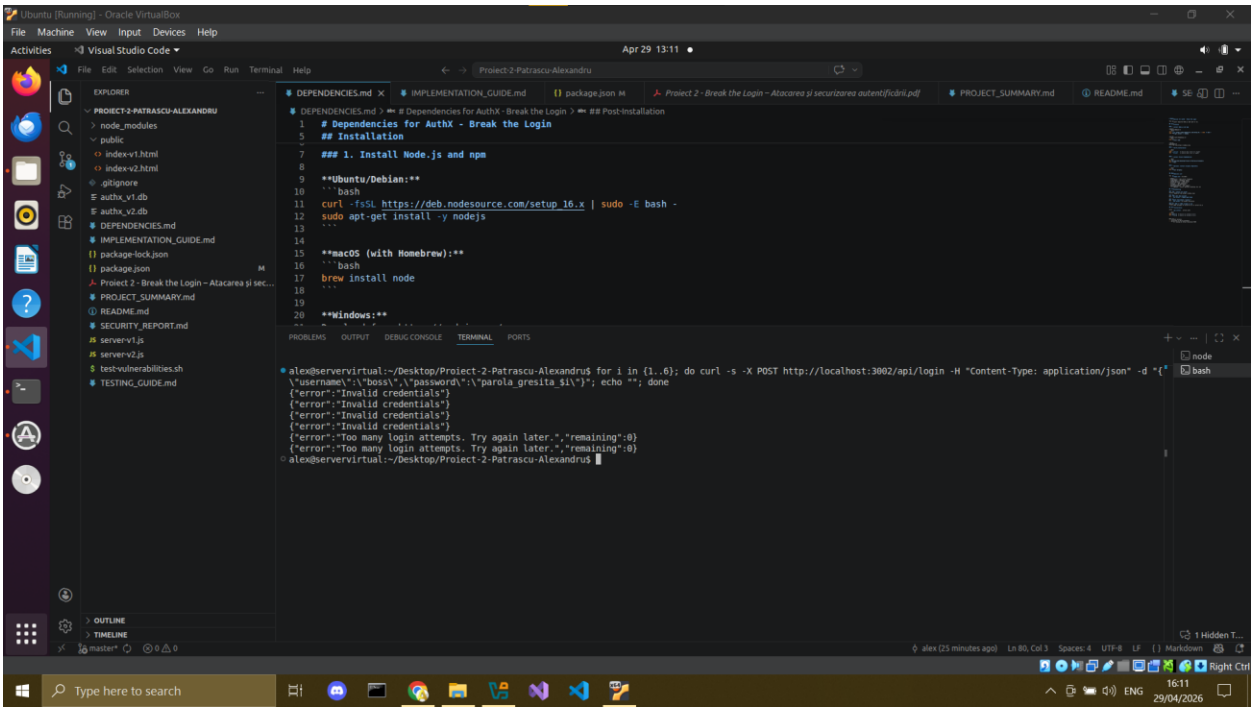
- Interogarea bazei de date securizate arată un cu totul alt peisaj pentru credențialele stocate. Parolele sunt acum salvate sub forma unor structuri complexe specifice algoritmului bcrypt, imposibil de decriptat direct.



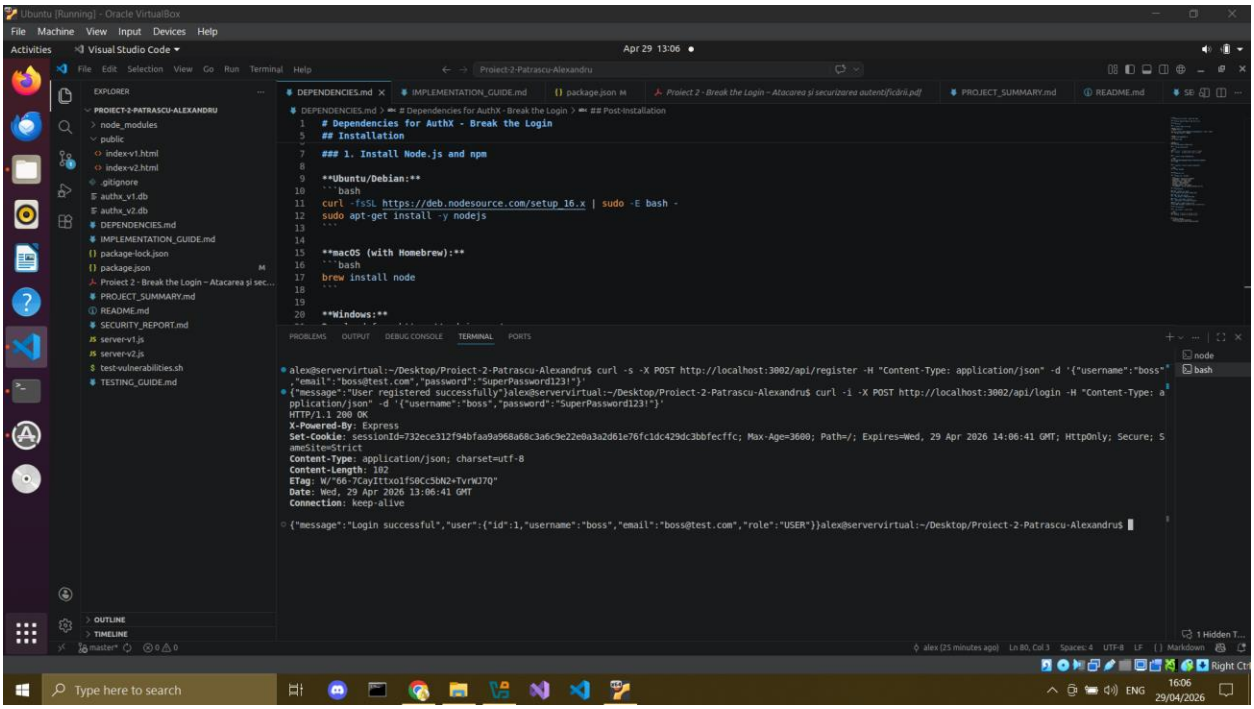
- Am verificat mecanismul anti enumerare trimițând din nou date greșite. Atât pentru utilizatorul inexistent, cât și pentru parola greșită, răspunsul a fost uniform, indicând generic faptul că datele de identificare sunt nevalide, fără a oferi detalii ajutătoare.



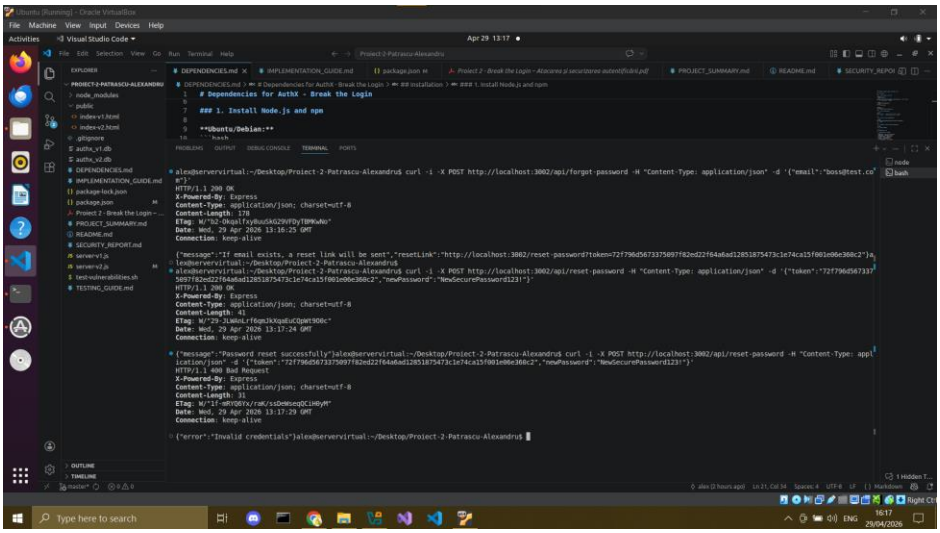
- Pentru a verifica limitarea ratei de cereri, am executat din nou scriptul buclă de atac. După procesarea a cinci cereri cu date incorecte, serverul a activat protecția și a început să returneze un mesaj specific de blocare a contului pentru tentative multiple.



- La o autentificare legitimă, analizarea anteturilor a relevat o schimbare majoră. Cookie-ul de sesiune este acum protejat cu toate directivele de securitate moderne, protejând datele împotriva interceptărilor și manipulărilor prin intermediul navigatorului web.

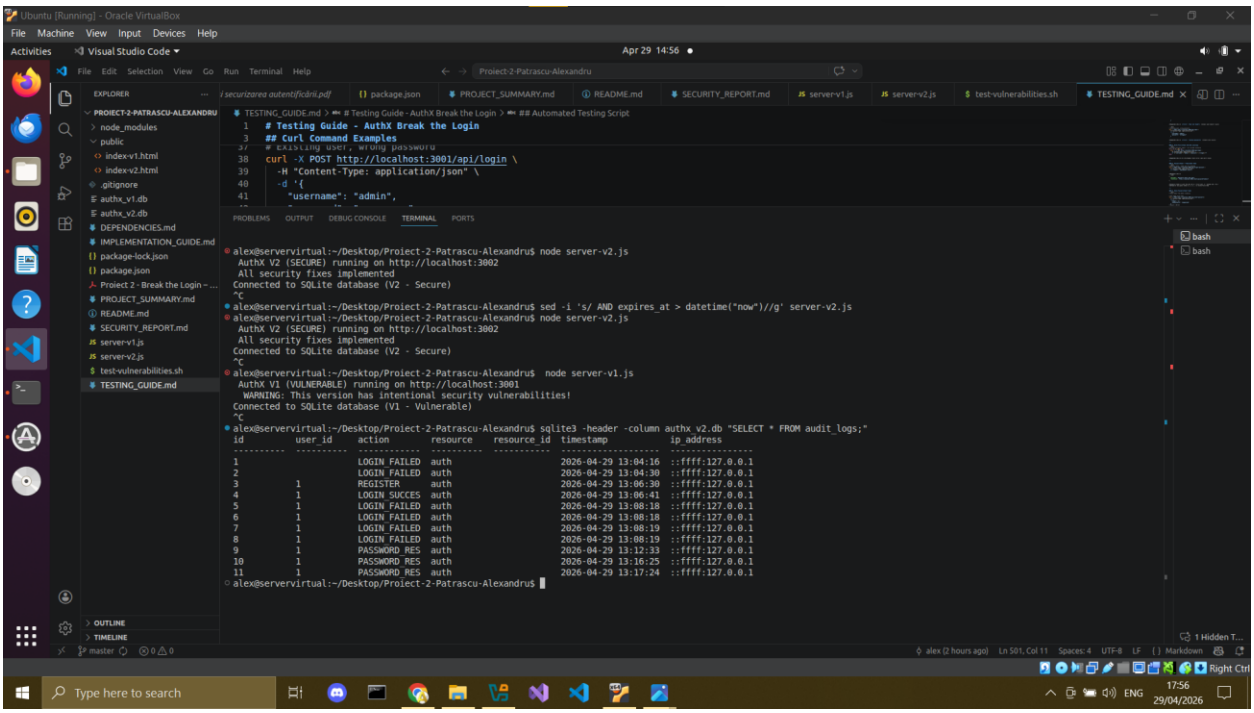


- În cele din urmă, am testat fluxul de resetare a parolei. Noul identificator primit a fost extrem de lung și complex. L-am folosit cu succes o singură dată pentru a modifica parola. O nouă cerere identică folosind același identificator a fost respinsă de server, demonstrând validitatea conceptului de utilizare unică a elementelor de securitate.



Audit și logging

- Verificarea acestui tabel arată o listă ordonată cronologic cu toate succesele și eșecurile operațiunilor de conectare și resetare, incluzând adresele de rețea. Această componentă asigură trasabilitatea completă și permite o eventuală analiză medico legală în cazul unui incident de securitate.



Concluzii

1. Experiența realizării acestui proiect mi-a oferit o perspectivă detaliată asupra simetriei dintre atac și apărare în contextul dezvoltării software.

2. Am învățat că o funcționalitate care din punct de vedere operațional își îndeplinește rolul poate fi profund vulnerabilă dacă nu se bazează pe o mentalitate proactivă de securizare.

3. Adoptarea practicilor sigure, cum ar fi validarea riguroasă a datelor de intrare, stocarea adecvată a elementelor criptografice și abstractizarea mesajelor de eroare destinate utilizatorilor finali, reprezintă un pilon fundamental pentru fiabilitatea oricărei aplicații expuse unui mediu de producție ostil.

4. Securitatea nu se poate adăuga superficial după finalizarea codului, ci trebuie să fie integrată la nivel de arhitectură.

Anexă Cod Sursă

Secțiunea Fișier server versiunea 1 (Vulnerabil) (Aici vei da Copy-Paste la tot codul din fișierul server-v1.js pe care ți l-am oferit anterior, pentru a demonstra logica defectuoasă.)

Secțiunea Fișier server versiunea 2 (Securizat) (Aici vei da Copy-Paste la tot codul din fișierul server-v2.js, pentru a demonstra aplicarea funcțiilor criptografice și a filtrelor de limitare.)